CyberLabX – Technical Architecture Document (TAD)

Version: 1.0

Project Type: SaaS (Educational Technology + Cybersecurity)

Architecture Style: Modular Monolith (Phase 1) → Microservices Ready (Future)

---

1. Architecture Philosophy

CyberLabX is not just a website.

It is composed of five major systems working together:

None

```
                    CyberLabX

                 React Frontend

                        |

               REST API Gateway

                        |

              Express Backend API

                        |

  ┌──────────────┬──────────────┬──────────────┐
  |              |              |              |
Auth         Experiment     Lab Engine    Reports
Service       Service         Service      Service
```

```
                    |
              MongoDB Database
                    |
          Docker Container Engine
                    |
     Kali + Target Machines Network
```

For Version 1, these services will exist as modules inside one Express application. That keeps deployment simple while allowing you to split them into independent services later if needed.

---

2. Technology Stack

Frontend

Technology	Reason
React 19	Component-based architecture, mature ecosystem
Vite	Extremely fast development and builds
TypeScript	Type safety, fewer runtime bugs, easier maintenance
Tailwind CSS	Rapid UI development with consistent design
React Router	Client-side routing
Axios	HTTP client
React Query (TanStack Query)	Server state management, caching, retries

xterm.js	Browser-based Linux terminal
Framer Motion	Smooth animations for the UI
Chart.js	Progress and analytics charts

Why React?

- Large ecosystem
- Easy hiring
- Highly maintainable
- Excellent for dashboards

Backend

Technology	Reason
Node.js	Non-blocking I/O, ideal for concurrent users
Express.js	Lightweight, flexible REST API framework
TypeScript	Safer backend development
Socket.IO	Real-time terminal communication
JWT	Stateless authentication
bcrypt	Password hashing
Multer	File uploads (future reports/assets)

Database

MongoDB

Why MongoDB?

CyberLabX stores documents like:

- Experiments
- Lab definitions

- Commands
- Reports

These are hierarchical and evolve over time. MongoDB's flexible schema makes experiment definitions much easier than a rigid relational model.

Lab Environment

Technology	Reason
Docker	Lightweight isolation
Docker Compose	Multi-container orchestration
Alpine Linux	Small helper images
Kali Linux	Student environment
Metasploitable / OWASP Juice Shop	Target systems for later labs

---

Authentication

JWT

```
None
Student Login

↓

JWT Generated

↓

Stored in HttpOnly Cookie

↓
```

```
Verified Every Request
```

Why JWT?

- Stateless
- Scalable
- Easy integration

Deployment

Initially

```
None
Ubuntu Server

↓

Docker

↓

Nginx

↓

Node

↓

MongoDB
```

Later

```
None
Kubernetes
```

3. Project Folder Structure

```
None
CyberLabX/

|

├── client/

|   ├── public/

|   ├── src/

|   |

|   ├── assets/

|   ├── components/

|   |       ├── common/

|   |       ├── terminal/

|   |       ├── dashboard/

|   |       ├── experiments/

|   |       └── ui/

|   |

|   ├── pages/

|   |       Login/
```

```
|   |      Dashboard/
|   |      Experiment/
|   |      Report/
|   |      Faculty/
|   |      Admin/
|   |
|   ├── layouts/
|   ├── hooks/
|   ├── services/
|   ├── contexts/
|   ├── routes/
|   ├── utils/
|   ├── types/
|   └── App.tsx
|
├── server/
|
|   ├── src/
```

```
|   |
|   ├── config/
|   ├── middleware/
|   ├── controllers/
|   ├── routes/
|   ├── services/
|   ├── repositories/
|   ├── models/
|   ├── sockets/
|   ├── docker/
|   ├── experiments/
|   ├── grading/
|   ├── reports/
|   ├── auth/
|   ├── utils/
|   └── server.ts
|
├── docker/
```

```
|
|   ├── kali/
|   ├── ubuntu/
|   ├── nmap/
|   ├── compose/
|   └── network/
|
├── docs/
├── scripts/
├── .env
├── docker-compose.yml
└── README.md
```

---

4. Backend Modules

None

```
Auth Module

Experiment Module

Lab Module

Terminal Module
```

```
Grading Module

Reports Module

Faculty Module

Admin Module

Analytics Module
```

Each module owns its own controllers, services, and models.

---

5. Database Schema

Users Collection

```
None
User

_id

firstName

lastName

email

passwordHash

role

department
```

```
college

profileImage

isVerified

createdAt

updatedAt
```

Role

```
None
Student

Faculty

Admin
```

Experiments

```
None
Experiment

_id

title

slug

description

difficulty
```

```
estimatedTime

status

createdBy

createdAt

updatedAt
```

Example

```
None
Network Mapping using Nmap
```

Experiment Sections

Instead of storing everything in one document:

```
None
ExperimentSection

_id

experimentId

type

content

order
```

Type

```
None
Theory

Procedure

Objective

Observation

Conclusion
```

This allows richer content and easier editing.

Tasks

```
None
Task

_id

experimentId

title

description

hint

expectedCommand

marks

sequence
```

Example

```
None
Task 1

Perform SYN Scan

Expected

nmap -sS
```

Student Attempts

```
None
Attempt

_id

studentId

experimentId

startTime

endTime

score

status
```

Status

```
None
Running

Completed
```

```
Expired
```

Command Logs

```
None
CommandLog

_id

attemptId

command

output

timestamp

success
```

This is valuable for grading, analytics, and debugging.

Quiz

```
None
Quiz

_id

experimentId

questions[]
```

Each question

```
None
Question

question

options

correctAnswer

explanation
```

Reports

```
None
Report

_id

student

experiment

score

remarks

pdfPath

generatedAt
```

Lab Sessions

```
None
LabSession
```

```
_id

attemptId

containerId

networkName

ipAddress

status

startedAt

endedAt
```

Notifications

None

```
Notification

_id

user

title

message

isRead

createdAt
```

Database Relationships

```
None
User

|

├──────────────┐

|              |

Attempt     Notification

|

|

Experiment

|

├──────────┐

|          |

Tasks    Quiz

|

Attempt

|

├────────────┐

|            |
```

```
CommandLog  Report

|

LabSession
```

---

6. API Structure

```
None
/api/auth

/api/users

/api/experiments

/api/tasks

/api/labs

/api/terminal

/api/quiz

/api/reports

/api/faculty

/api/admin
```

Example REST Endpoints

Authentication

None

```
POST /login

POST /register

POST /logout

GET /me
```

Experiments

None

```
GET /experiments

GET /experiments/:id

POST /experiments

PUT /experiments/:id

DELETE /experiments/:id
```

Lab

None

```
POST /labs/start

POST /labs/stop

GET /labs/status
```

Terminal

```
None
Socket.IO

↓

execute-command

↓

receive-output
```

Docker Architecture

```
None
Docker Host

|

├── MongoDB

├── Node

├── React

├── Nginx

|

└── Student Containers

      ├── Student 1

      ├── Student 2
```

```
      ├── Student 3
      └── Student N
```

Each student receives a fresh, isolated container.

Networking

```
None
Student Browser
↓
Node Backend
↓
Docker API
↓
Kali Container
↓
Internal Network
↓
Target Machine
```

The container cannot reach the public Internet by default.

Security

- JWT in HttpOnly cookies.
- Passwords hashed with bcrypt.
- Role-Based Access Control (RBAC).
- Rate limiting on authentication endpoints.
- Input validation using Zod or Joi.
- CSRF protection if using cookies.
- Container resource limits (CPU, memory).
- Network isolation for lab containers.
- Audit logs for faculty/admin actions.

Environment Variables

Backend

```
None
NODE_ENV=development
PORT=5000

CLIENT_URL=http://localhost:5173

MONGODB_URI=mongodb://localhost:27017/cyberlabx

JWT_SECRET=your-long-random-secret
JWT_EXPIRES_IN=7d

BCRYPT_ROUNDS=12

DOCKER_SOCKET=/var/run/docker.sock
DOCKER_NETWORK=cyberlabx_network

LAB_IMAGE=cyberlabx-kali:latest
```

```
TARGET_IMAGE=cyberlabx-target:latest

REPORT_STORAGE=./reports

LOG_LEVEL=info
```

Frontend

```
None
VITE_API_URL=http://localhost:5000/api
VITE_SOCKET_URL=http://localhost:5000
VITE_APP_NAME=CyberLabX
```

Configuration Notes

- **Docker permissions:** The backend will need access to the Docker Engine to create and manage lab containers. This is powerful and should be carefully restricted in production.
- **Resource limits:** Configure CPU and memory limits for each student container to prevent a single user from exhausting server resources.
- **Networking:** Place lab containers on an isolated Docker network with no outbound Internet access unless an experiment explicitly requires it.
- **Secrets:** Never commit `.env` files to version control. Provide a `.env.example` with placeholder values.
- **Logging:** Centralize logs for the backend and lab management to simplify troubleshooting.
- **Backups:** Schedule regular backups of MongoDB and generated reports.

Future Scalability

The architecture is intentionally designed so that modules such as **Authentication**, **Lab Engine**, **Grading**, and **Reports** can be extracted into separate microservices later without changing the frontend or API contracts. The Docker-based lab engine can also be replaced with

a Kubernetes-based orchestration layer if the platform needs to support hundreds or thousands of concurrent students.

One architectural recommendation

For the MVP, **avoid introducing Kubernetes, Redis, message queues, or multiple databases**. Those technologies add operational complexity without solving an immediate problem. A stack of **React + Express + MongoDB + Docker + Socket.IO** is sufficient for a robust first version and will let you focus on delivering the educational experience rather than managing infrastructure. Once the platform demonstrates value and concurrent usage grows, you can incrementally introduce caching, background job processing, and container orchestration.